EXP 1 -Write a python program to implement Simple Calculator program?

AIM

To write a Python program to implement a simple calculator that performs addition, subtraction, multiplication, and division.

ALGORITHM

1. Start the program.
2. Get two numbers from the user.
3. Display the arithmetic operations: addition, subtraction, multiplication, and division.
4. Get the user's choice.
5. Perform the selected operation using conditional statements.
6. Display the result.
7. Stop the program.

CODE

```
# Simple Calculator Program

a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

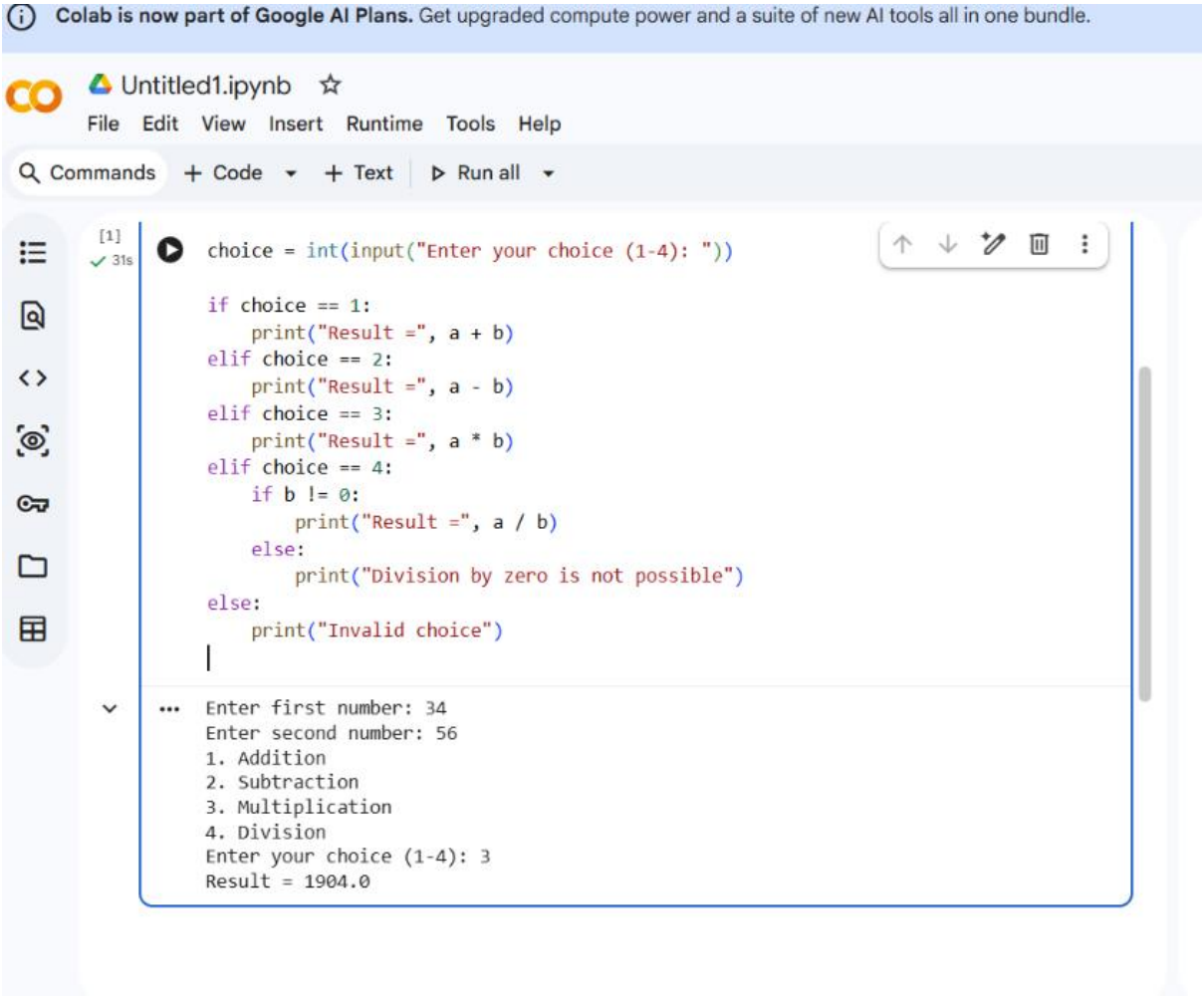
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")

choice = int(input("Enter your choice (1-4): "))

if choice == 1:
    print("Result =", a + b)
```

```
elif choice == 2:
    print("Result =", a - b)
elif choice == 3:
    print("Result =", a * b)
elif choice == 4:
    if b != 0:
        print("Result =", a / b)
    else:
        print("Division by zero is not possible")
else:
    print("Invalid choice")
```

OUTPUT



2. Python Program to Add Two Matrices

AIM

To write a Python program to add two matrices.

ALGORITHM

1. Start the program.
2. Get two matrices as input from the user.
3. Add the corresponding elements of both matrices.
4. Display the resulting matrix.
5. Stop the program.

CODE

```
# Add Two Matrices

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = [[0, 0], [0, 0]]

for i in range(2):
    for j in range(2):
        C[i][j] = A[i][j] + B[i][j]

print("Matrix A:", A)
print("Matrix B:", B)
print("Sum of matrices:")
for row in C:
    print(row)
```

OUTPUT:

```
print("Matrix A:", A)
print("Matrix B:", B)
print("Sum of matrices:")
for row in C:
    print(row)
```

```
Matrix A: [[1, 2], [3, 4]]
Matrix B: [[5, 6], [7, 8]]
Sum of matrices:
[6, 8]
[10, 12]
```

3. Python Program to Transpose a Matrix

AIM

To write a Python program to find the transpose of a matrix.

ALGORITHM

1. Start the program.
2. Define a matrix.
3. Find the transpose by interchanging rows and columns.
4. Display the transposed matrix.
5. Stop the program.

CODE

```
# Transpose a Matrix

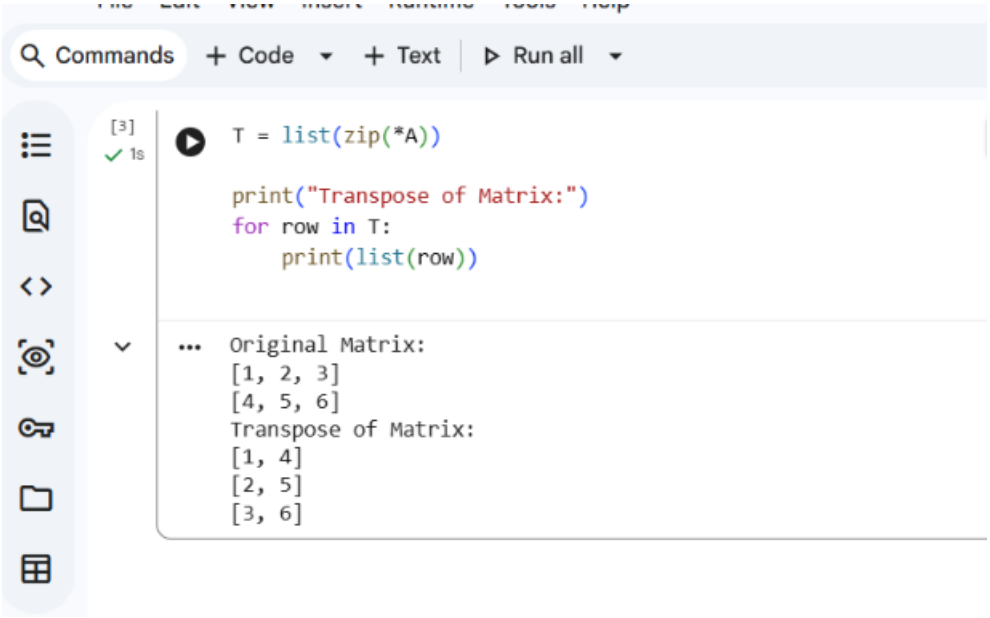
A = [[1, 2, 3], [4, 5, 6]]

print("Original Matrix:")
for row in A:
    print(row)

T = list(zip(*A))
```

```
print("Transpose of Matrix:")
for row in T:
    print(list(row))
```

OUTPUT:



4. Python Program to Sort a Sentence in Alphabetical Order

AIM

To write a Python program to sort the words in a sentence in alphabetical order.

ALGORITHM

1. Start the program.
2. Get a sentence from the user.
3. Split the sentence into individual words.
4. Sort the words alphabetically.
5. Display the sorted words.
6. Stop the program.

CODE

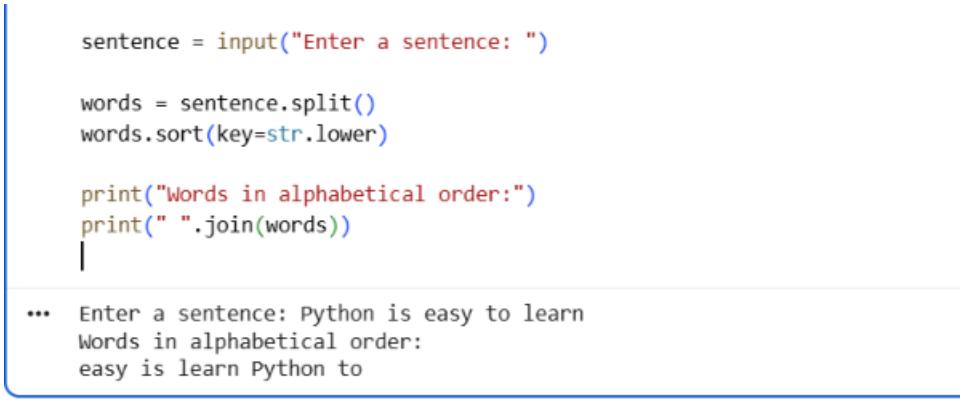
```
# Sort a Sentence Alphabetically
```

```
sentence = input("Enter a sentence: ")

words = sentence.split()
words.sort(key=str.lower)

print("Words in alphabetical order:")
print(" ".join(words))
```

OUTPUT:



5. Python Program to Implement List Operations

AIM

To write a Python program to demonstrate list operations such as nested list, length, concatenation, membership, iteration, indexing, and slicing.

ALGORITHM

1. Start the program.
2. Create two lists and a nested list.
3. Find the length of a list using len().
4. Join two lists using +.
5. Check membership using in.
6. Display elements using iteration.
7. Access elements using indexing.
8. Extract elements using slicing.

9. Stop the program.

CODE

```
# List Operations

L1 = [10, 20, 30, 40]
L2 = [50, 60]

# Nested List
nested = [1, 2, [3, 4]]
print("Nested List:", nested)

# Length
print("Length:", len(L1))

# Concatenation
print("Concatenation:", L1 + L2)

# Membership
print("20 in L1:", 20 in L1)

# Iteration
print("Iteration:")
for item in L1:
    print(item)

# Indexing
print("First Element:", L1[0])

# Slicing
```

print("Slicing:", L1[1:3])

```
# Slicing
print("Slicing:", L1[1:3])
```

```
Nested List: [1, 2, [3, 4]]
Length: 4
Concatenation: [10, 20, 30, 40, 50, 60]
20 in L1: True
Iteration:
10
20
30
40
First Element: 10
Slicing: [20, 30]
```